Lab 10 Batch G1

Dated 19-10-2026

(Due date: 21.10.2026)

1. Write a LEX and YACC program to convert an infix arithmetic expression into its equivalent postfix expression.

Your program must support integer operands and the operators +, -, *, /, and ^, along with parentheses.

Requirements:

1. Use LEX to recognize integer numbers, arithmetic operators, parentheses, and whitespace.
2. Use YACC to define grammar productions that correctly represent operator precedence and associativity.
3. Use semantic actions to generate the postfix expression.
4. Ensure that multiplication and division have higher precedence than addition and subtraction, and that exponentiation has higher precedence than multiplication and division.
5. Handle parentheses correctly and display an appropriate error message for invalid expressions

Test inputs and expected outputs:

Infix input	**Postfix output**
5+3*2	5 3 2 * +
(5+3)*2	5 3 + 2 *
8-4/2	8 4 2 / -
2^3^2	2 3 2 ^ ^

Instructions to students: How to implement YACC

Step 1: Install the required tools

Use a Linux terminal or Ubuntu environment. Install GCC, Flex, and Bison (a YACC-compatible parser generator).

```
sudo apt update

sudo apt install gcc flex bison
```

The uploaded tutorial demonstrates the traditional yacc and lex commands and explains how the parser and scanner are compiled and linked together

Step 2: Create two source files

Create the following files in the same directory:

- program.l — LEX file for token recognition.
- program.y — YACC file for grammar rules and semantic actions.

The LEX file (program.l) must:

- Include the generated parser header, y.tab.h.
- Recognize numbers and identifiers using regular expressions.
- Return tokens for numbers and identifiers to YACC.
- Return operators and parentheses as character tokens.
- Ignore spaces, tabs, and newlines where appropriate.
- Handle invalid characters.

The YACC file (program.y) must:

- Declare tokens and operator precedence.
- Define grammar productions.
- Include semantic actions for conversion, evaluation, or code generation.
- Define yyerror() to report syntax errors.
- Call yyparse() from main().

Step 3: Define precedence in YACC

The following declarations illustrate the precedence order:

```
%left '+' '-'

%left '*' '/'

%right '^'

%right UMINUS
```

YACC assigns higher precedence to declarations appearing later. UMINUS is a precedence marker for unary minus; the corresponding grammar production must use %prec UMINUS.

For example: Yacc

```
expr:
    '-' expr %prec UMINUS
  | expr '+' expr
  | expr '*' expr
```

;

Students must complete the grammar and semantic actions for their assigned question. The exponentiation operator is right-associative, so 2^3^2 is interpreted as 2^(3^2).

Step 4: Generate the parser and scanner

For traditional lex and yacc commands, execute:

```
yacc -d program.y
lex program.l
gcc y.tab.c lex.yy.c -o program -ll
```

If your system uses Flex and Bison, execute:

```
bison -dy program.y
flex program.l
gcc y.tab.c lex.yy.c -o program -lfl
```

Use the command set supported by your installed environment.

Step 5: Run and test the program

```
./program
```

Do not submit only the output. The implementation must use both LEX and YACC, and the semantic actions must perform the actual conversion, evaluation, or intermediate-code generation. Students may use online documentation to install and troubleshoot the tools, but must be able to explain their own code during the lab evaluation.

Instructions

Due date: 21.10.2026, 11:30 A.M for Batch G1 (30 minutes extra is given for the documentation)

Cut-off date: 22.10.2026, 11:59 P.M. (Late submission deadline)

Note:

1. If the submission is after the due date/time, marks will be deducted for every 6 hours till the cut-off time. After the Cut-off date, No evolution and no marks
2. If the output is not shown for any of the above components, the mark cannot be given even if it is included in your documentation.